

TRAITS & IMPLEMENTATIONS

Definition

A **trait** in Rust is like an **interface** in other languages. It defines **shared behavior** that types can implement. Traits allow you to write **generic code** that works with *any* type that implements a specific set of capabilities.

« FLY » TRAIT



Plane



Bird



Superman





« A Trait is a **contract** that describes functionality that a type **must have**. »

We were already using it :)



```
#[derive(Debug)]  
enum MyError {  
    DivisionByZero,  
    FileNotFound,  
}
```

vs Debug **vs** Display

Trait	Format macro	Output style	Use case
Debug	{:?}	Developer-focused	Logs, debugging info
Display	{}	User-friendly	End-user messages, UI

Exercise 1: Reminder Default Trait



Example



```
trait Speak {  
    fn speak(&self) -> String;  
}  
  
struct Dog;  
  
impl Speak for Dog {  
    fn speak(&self) -> String {  
        "Woof!".to_string()  
    }  
}
```


Exercise 2: Traits Animals

The Bound Trait



```
fn shout<T: Display>(value: T) {  
    println!("HEY! {}", value);  
}
```

The Bound Trait

```
● ● ●  
  
use std::fmt::Display;  
  
// This function accepts any type T –  
// BUT ONLY if T implements the Display trait  
fn shout<T: Display>(value: T) {  
    println!("HEY! {}", value);  
}  
  
fn main() {  
    shout("hello");    // Works: &str implements Display  
    shout(42);         // Works: i32 implements Display  
  
    // shout(vec![1, 2, 3]); // ✗ Error: Vec<i32> doesn't implement Display  
}
```


Exercise 3: Bound Trait

The where clause

The **where clause** lets you write trait bounds **below** the function for **better readability**, especially with multiple traits or types. It works the same as inline bounds, just cleaner.

The where clause



```
fn do_stuff<T: Clone + std::fmt::Debug>(value: T) {  
    println!("{:?}", value.clone());  
}
```

1) 🙅 Without where clause



```
fn do_stuff<T>(value: T)  
where  
    T: Clone + std::fmt::Debug,  
{  
    println!("{:?}", value.clone());  
}
```

2) ✅ With where clause

Exercise 4: The Where clause

Your turn!

YOUR TEXT GOES HERE



```
trait Describable {  
    fn describe(&self) -> String;  
}  
  
struct Person {  
    name: String,  
    age: u8,  
}  
  
impl Describable for Person {  
    fn describe(&self) -> String {  
        format!("{}", self.name, self.age)  
    }  
}
```


YOUR TEXT GOES HERE